

TCN Hardware Implementation Plan

Author	Ryan Gaffere
Version	v1.0
Date	May 8th, 2026

This document provides details on the hardware implementation of the TCN Accelerator.

High-Level Architecture

The accelerator is organized into four major modules:

1. I/O Interface
2. Memory
3. Model Architecture
4. Cache

The system performs streaming inference on 6-channel IMU input data. At each inference step, the accelerator receives the latest sample across all 6 channels, updates the internal cache state, runs the TCN model, and outputs the latest normalized prediction along with an anomaly flag.

The architecture targets deterministic, low-latency neural network inference using a fully spatial residual-block pipeline and dedicated MAC resources per residual block.

I/O Interface

The accelerator uses AXI4-Stream for runtime input and output communication.

Inputs

- Latest 6-channel IMU sample
 - Accelerometer X/Y/Z
 - Gyroscope X/Y/Z

Outputs

- Latest normalized 6-channel prediction
- Anomaly flag

The anomaly flag is generated by comparing the predicted output against the incoming sensor sample and determining whether the prediction error exceeds a configured threshold.

Weights and biases are loaded from flash memory during boot initialization and stored in SRAM for runtime inference.

The design targets INT8 quantized inference, since INT8 significantly reduces:

- memory footprint
- SRAM bandwidth
- MAC complexity
- and overall hardware area

compared to floating-point arithmetic.

Memory

The accelerator uses SRAM-based memory rather than FPGA BRAM, since the target implementation is ASIC-oriented.

SRAM is used because:

- it provides low-latency access
- supports high-bandwidth parallel reads
- and is well suited for repeated streaming convolution operations

Memory stores:

- model weights
- biases
- cache state
- intermediate activations
- and runtime data required for inference

Weights are static during inference and are loaded once during boot initialization.

The trained floating-point weights are quantized before hardware deployment. The target format is:

- INT8 weight storage
- INT8 activation storage
- Wider MAC accumulation datatype during convolution

The MAC datapath will use a wider internal accumulator, likely INT32, to prevent overflow during multiply-accumulate operations before requantization back to INT8.

The primary memory concern is bandwidth rather than capacity, since many MAC units operate concurrently across the residual-block pipeline.

Model Architecture

The network uses a fixed Temporal Convolutional Network (TCN) architecture with reconfigurable weights.

Network Structure

- 6 input channels
- 7 residual blocks
- 2 causal convolutions per residual block
- Kernel size = 3
- Dilation schedule:
1, 2, 4, 8, 16, 32, 64

The receptive field is:

$$\text{RF} = 1 + 2 \times (k - 1) \times \sum(\text{dilation})$$

$$\text{RF} = 509$$

The cache stores 512 samples per channel to simplify circular indexing using a power-of-two buffer size.

MAC Architecture

Each residual block contains dedicated compute hardware.

Per residual block:

- 32 MAC units total
- Organized as:
 - 16-wide MAC bank for Conv1
 - 16-wide MAC bank for Conv2

Across the hidden network:

- $32 \text{ MACs} \times 7 \text{ residual blocks} = 224 \text{ MAC units}$

The design intentionally spends area on acceleration. Since this is a neural network accelerator, throughput improvement from dedicated compute hardware is prioritized over minimizing MAC count.

The architecture avoids full kernel unrolling, since adding additional MAC units for every kernel tap would produce diminishing throughput returns while significantly increasing area and routing complexity.

Pipelining Strategy

The accelerator uses a fully spatial residual-block pipeline.

Each residual block has dedicated compute hardware and can operate concurrently with other residual blocks once the pipeline is filled.

Instead of reusing one MAC bank across all hidden layers, every residual block remains active during streaming inference.

Example execution after pipeline fill:

```
Step N:      B1(sample t)
Step N+1:    B2(sample t)
             B1(sample t+1)
Step N+2:    B3(sample t)
             B2(sample t+1)
             B1(sample t+2)
```

Within each residual block:

- Conv2 begins once Conv1 produces valid intermediate outputs
- Valid signals propagate between convolution stages
- Valid signals propagate between residual blocks

This allows downstream blocks to begin processing immediately after upstream outputs become available.

The result is a deeply pipelined streaming architecture with high throughput and deterministic execution.

Cache

The cache stores:

- streaming input history,
- hidden-layer history,
- and intermediate activations required for causal convolution.

The cache uses a ring buffer architecture to achieve $O(1)$ streaming updates.

Instead of shifting all history values on every new sample, the system advances a circular head pointer through SRAM.

This eliminates expensive memory movement operations and allows constant-time cache updates regardless of receptive field size.

Cache indexing:

```
cache_index = (head - offset) mod 512
```

Since the buffer size is a power of two, modulo operations can be implemented using bit masking instead of division.

Precomputed Dilation Offsets

The dilation schedule is fixed:

```
1, 2, 4, 8, 16, 32, 64
```

Because these dilation values never change, the delayed tap offsets for every residual block are precomputed.

For kernel size 3, each convolution requires:

```
tap 0: t
tap 1: t - dilation
tap 2: t - 2*dilation
```

Instead of dynamically counting backward through memory during runtime, each residual block directly uses precomputed offsets.

Example:

```
cache_index = (head - precomputed_offset) & 511
```

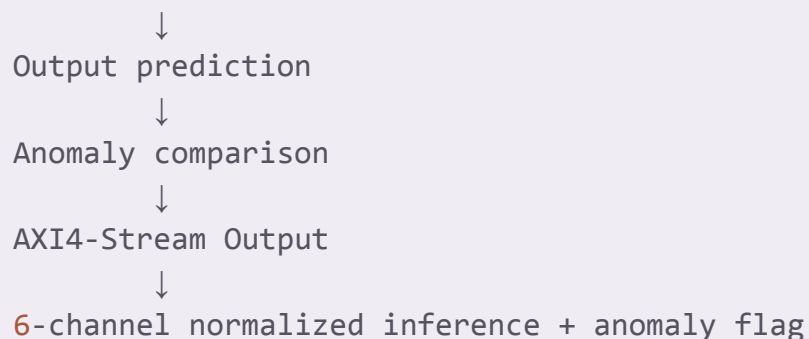
This improves the architecture by:

- reducing address-generation complexity,
- reducing runtime iteration,
- simplifying SRAM access,
- improving timing predictability,
- and reducing cache-controller overhead.

The cache architecture is designed specifically for efficient streaming inference and deterministic memory access patterns.

Overall Dataflow

```
6-channel input sample
    ↓
AXI4-Stream Input
    ↓
Ring-buffer cache update
    ↓
Residual Block 1
    ↓
Residual Block 2
    ↓
Residual Block 3
    ↓
Residual Block 4
    ↓
Residual Block 5
    ↓
Residual Block 6
    ↓
Residual Block 7
```



This architecture prioritizes:

- deterministic streaming inference,
- high throughput,
- efficient SRAM access,
- spatial pipelining,
- $O(1)$ cache updates,
- realistic area/performance tradeoffs,
- and ASIC-oriented accelerator design principles.

Implementation Order

The accelerator will be implemented in a bottom-up order so each module can be independently tested before full network integration.

1. MAC Unit

- Implement INT8 multiply with wider accumulation.
- Verify against simple known input/weight cases.

2. ReLU / Activation Unit

- Implement activation behavior after convolution.
- Verify positive, negative, and zero cases.

3. Ring Buffer Cache

- Implement $O(1)$ circular buffer updates.
- Verify head pointer movement and history storage.

4. Dilation Addressing

- Add precomputed dilation offsets.
- Verify tap reads for each dilation value.

5. Single Convolution Stage

- Combine MAC, ReLU, cache reads, and dilation addressing.
- Verify against the C++ golden model for one convolution.

6. **Residual Block**

- Implement Conv1, Conv2, ReLU, residual add, and valid propagation.
- Verify one residual block against the software reference.

7. **Hidden Layer / Block Pipeline**

- Connect residual blocks through valid-signal propagation.
- Verify staged execution and cache writeback between blocks.

8. **Full TCN Network**

- Integrate all 7 residual blocks.
- Verify full-network inference against the C++ reference output.

9. **Anomaly Detection Logic**

- Add prediction-error comparison and anomaly flag generation.
- Verify threshold behavior.

10. **AXI4-Stream Interface**

- Add streaming input/output interface.
- Verify sample ingestion, output formatting, and valid/ready behavior.

11. **System Integration and Verification**

- Run full hardware/software validation.
- Compare RTL output against the C++ golden model.
- Measure latency, throughput, SRAM usage, and resource utilization.